Architecture Decision Document

This document builds collaboratively through step-by-step discovery. Sections are appended as we work through each architectural decision together.

Project Context Analysis

Requirements Overview

Functional Requirements: 46 FRs across 11 capability areas: Authentication & User Management (FR1–4), Timesheet & Time Tracking (FR5–10), Expense Tracking (FR11–14), Project Management (FR15–19), Staffing & Availability (FR20–26), AI-Assisted Staffing (FR27–28), PTO Management (FR29–31), Client Management (FR32–35), Reporting (FR36–40), Invoice Generation (FR41–44), Data Migration (FR45–46).

Architectural implication: modules are tightly coupled via state events — availability, timesheets, staffing needs, and PTO all share a propagation dependency. The event pipeline is the architectural spine of the system.

Non-Functional Requirements:

- Performance: ≤2s page loads, ≤3s availability calendar render, ≤5s event propagation, ≤5s report queries (5-year dataset), ≤10s invoice generation
- Security: TLS 1.2+, SSO-only auth, server-side RBAC, write-once audit trail, 30-min idle session timeout, malware scan on receipt uploads
- Reliability: 99.5% uptime (business hours), durable write-before-confirm, auto-retry with admin visibility after 3 failures
- Scalability: 70–100 concurrent users at launch; horizontal scaling to 200+ without schema or service redesign

Scale & Complexity:

- Primary domain: Full-stack web platform
- Complexity level: Medium-High
- Estimated architectural components: 8–10 (API gateway, auth service, core domain modules ×6, event bus, AI scoring module, migration pipeline)

Technical Constraints & Dependencies

- Frontend: JavaScript SPA (React or similar)
- Backend: C# REST API
- Auth: OAuth 2.0 / OIDC via SSO provider (Google Workspace / Azure AD / Microsoft Entra — TBD at deployment)
- Database: Relational (joins across timesheets, projects, clients, employees, invoices required)
- State propagation: Event-driven via transactional outbox pattern
- Growth integration: QuickBooks Online API (Phase 2 only)
- Migration: One-time Excel (.xlsx) import at launch

Cross-Cutting Concerns Identified

1. **Event propagation pipeline** — must be reliable, retryable, and observable; failure path requires admin visibility
2. **RBAC enforcement** — server-side on every protected endpoint; UI-only enforcement is a security violation per NFRs
3. **Audit logging** — write-once, tamper-evident entries for timesheet changes and billed-hour modifications; append-only write pattern required
4. **Session management** — JWT tokens with 30-min idle expiry; SSO re-auth flow
5. **File handling** — receipt upload pipeline with malware scanning before storage
6. **AI scoring layer** — must degrade gracefully to availability-only ranking when historical data threshold not met (❤️ completed assignments)

Architectural Decision Records (Pre-committed)

ADR-01 — Event Propagation: Transactional outbox pattern. State changes and their events write atomically to the DB in one transaction; a background worker dispatches and retries. Single deployable, guaranteed delivery, admin-visible retry queue. Event contracts designed for broker migration without touching producers or consumers.

ADR-02 — RBAC Enforcement: Two-layer server-side enforcement. Layer 1: policy middleware on every API route (role capability check). Layer 2: service-layer ownership check (user-scoped resource access). UI mirrors enforcement for UX only — server never trusts UI state.

ADR-03 — Audit Trail: Append-only audit log table. Application DB user has INSERT-only permission on audit tables — no UPDATE or DELETE. Invoice immutability enforced at service layer; mutation attempts rejected before reaching the DB. Timesheet current state is mutable pre-invoice; immutable post-invoice-generation.

ADR-04 — AI Scoring Layer: Isolated module within the C# API behind a clean `IStaffingRecommendationService` interface. Fallback logic (❤️ completed assignments → availability-only ranking) lives inside the module. Extractable to a standalone service in Phase 2 without changing callers.

Starter Template Evaluation

Primary Technology Domain

Full-stack web platform — decoupled frontend SPA and backend REST API. Not a monorepo full-stack framework; frontend and backend are separate deployables.

Selected Starters

Frontend: Modern React Template 2026

Rationale: TanStack Query handles API caching and polling for the availability calendar and reporting views. TanStack Router provides type-safe navigation across the multi-module layout. Vitest + Playwright covers unit and e2e testing needs. WCAG 2.2 AA accessibility included.

Initialization:

```
# Clone from github.com/asudbury/modern-react-template
npm install && npm run dev
```

Architectural Decisions Provided:

- Language & Runtime: TypeScript strict mode, React 19
- Routing: TanStack Router (type-safe, file-based)
- Server State: TanStack Query (caching, polling, background refetch)
- Styling: TailwindCSS
- Testing: Vitest (unit), Playwright (e2e)
- Linting: ESLint + Prettier pre-configured
- Accessibility: WCAG 2.2 AA baseline

Backend: CleanArch.StarterKit v9.0.1 (.NET 9)

Rationale: Hangfire background jobs maps directly to the transactional outbox worker (ADR-01). Built-in audit logging maps to ADR-03. CQRS/MediatR separates read-heavy reporting queries from write-heavy timesheet and event commands. JWT auth aligns with SSO token validation requirement.

Initialization:

```
dotnet new install CleanArch.StarterKit::9.0.1
dotnet new cleanarch --name Ops
```

Architectural Decisions Provided:

- Language & Runtime: C# / .NET 9
- Architecture Pattern: Clean Architecture (Domain → Application → Infrastructure → WebApi)
- Command/Query: CQRS via MediatR
- Background Jobs: Hangfire (outbox worker, retry queue)
- Audit Logging: Built-in (maps to ADR-03 append-only audit trail)
- Auth: JWT middleware (configured for SSO token validation)
- Validation: FluentValidation
- Health Checks: Built-in endpoint
- ORM: EF Core (relational DB, complex joins for reporting)

Note: Project initialization using these commands should be the first implementation stories.

Core Architectural Decisions

Decision Priority Analysis

Critical Decisions (Block Implementation):

- Database: SQL Server with EF Core (Npgsql provider)

- Auth: JWT via SSO (OAuth 2.0 / OIDC) — no local credential storage
- Event propagation: Transactional outbox pattern (Hangfire worker)
- RBAC: Two-layer server-side enforcement

Important Decisions (Shape Architecture):

- File storage: Local filesystem / mapped volume (MVP)
- Caching: IMemoryCache on API server (MVP)
- UI state: Zustand
- API error format: RFC 7807 Problem Details
- API versioning: URL-based (`/api/v1/`)

Deferred Decisions (Pre-Deployment):

- Hosting target and cloud provider
- CI/CD pipeline tooling
- Structured logging / monitoring platform

Data Architecture

- **Database:** SQL Server
- **ORM:** EF Core (provided by CleanArch starter) — Microsoft.Data.SqlClient provider
- **Migrations:** EF Core Migrations — code-first schema management
- **Caching:** `IMemoryCache` on the API server for availability calendar snapshots and frequently-read reference data (clients, projects); revisit with distributed cache (Redis) if horizontal scaling required in Phase 2
- **Audit tables:** INSERT-only via application DB user; no UPDATE or DELETE permissions granted on audit log tables (enforces ADR-03)

Authentication & Security

- **Auth flow:** OAuth 2.0 / OIDC via company SSO provider (Google Workspace / Azure AD / Microsoft Entra — TBD at deployment); JWT bearer tokens validated on every API request
- **Session expiry:** 30-minute idle timeout enforced in JWT middleware; re-authentication via SSO redirect
- **RBAC enforcement:** Policy middleware (route-level capability check) + service-layer ownership check (row-level user-scoped access) — never UI-only
- **File storage:** Local filesystem / mapped volume for receipt attachments (MVP); malware scan hook at upload boundary before write; path to blob storage (Azure Blob / S3) available for Phase 2 horizontal scaling
- **Transport security:** TLS 1.2+ enforced at API gateway / reverse proxy layer

API & Communication Patterns

- **API style:** REST — resource-oriented endpoints, standard HTTP verbs
- **Versioning:** URL-based (`/api/v1/`) — simple, visible, compatible with all clients
- **Error format:** RFC 7807 Problem Details (`application/problem+json`) — native in .NET 9 via `IProblemDetailsService`; consistent machine-readable error bodies across all endpoints
- **Documentation:** Swagger / OpenAPI (provided by CleanArch starter) — auto-generated from controllers

- **Event communication:** Internal only via transactional outbox + Hangfire worker; no external message broker at MVP

Frontend Architecture

- **Framework:** React 19 + Vite + TypeScript strict mode
- **Routing:** TanStack Router (type-safe, file-based route definitions)
- **Server state:** TanStack Query (API caching, background refetch, polling for availability calendar updates)
- **UI state:** Zustand — global ephemeral state for modals, sidebar, multi-step staffing flow, notification queue for event propagation alerts
- **Styling:** TailwindCSS
- **Component approach:** Small reusable components; shadcn/ui as component primitive library
- **Testing:** Vitest (unit), Playwright (e2e)
- **Accessibility:** WCAG 2.2 AA baseline (provided by template)

Infrastructure & Deployment

- **Status:** Deferred — hosting target, CI/CD tooling, and monitoring platform to be decided before first deployment sprint
- **Constraints:** Architecture must support containerized (Docker) deployment to preserve hosting flexibility
- **Logging:** Structured logging via .NET `ILogger` with Serilog sink; destination (console / Seq / cloud) determined when infrastructure is decided

Decision Impact Analysis

Implementation Sequence:

1. Scaffold both projects (frontend + backend starters)
2. Configure SQL Server + EF Core connection + initial migration
3. Implement SSO JWT middleware + RBAC policy setup
4. Build transactional outbox infrastructure (Hangfire + outbox table)
5. Implement append-only audit log table + INSERT-only DB user
6. Build domain modules against established infrastructure
7. Wire Zustand stores + TanStack Query hooks per module

Cross-Component Dependencies:

- Event outbox must exist before any module that triggers state changes (staffing, PTO, availability)
- RBAC policies must be defined before any protected endpoint is built
- Audit log table must exist before timesheet write operations are implemented
- File upload pipeline (with malware scan hook) must be ready before expense submission is built

---

Implementation Patterns & Consistency Rules

Naming Patterns

Database Naming (SQL Server + EF Core):

- Tables: PascalCase plural — `TimesheetEntries`, `Projects`, `StaffingNeeds`
- Columns: PascalCase — `EmployeeId`, `CreatedAt`, `IsBillable`
- Primary keys: `Id` (Guid preferred for portability)
- Foreign keys: `[ReferencedEntity]Id` — `ProjectId`, `ClientId`, `EmployeeId`
- Audit table naming: `[Entity]AuditLog` — `TimesheetEntryAuditLog`
- Indexes: `IX_[Table]_[Column(s)]` — `IX_TimesheetEntries_EmployeeId_WeekStartDate`
- Outbox table: `OutboxMessages`

API Naming (REST + C#):

- Endpoints: plural kebab-case — `/api/v1/timesheet-entries`, `/api/v1/staffing-needs`
- Route parameters: `{id}` — `/api/v1/projects/{id}/members`
- Query parameters: camelCase — `?employeeId=&startDate=&endDate=`
- JSON field names: camelCase (default .NET System.Text.Json serialization)
- CQRS command/query naming: `[Action][Entity]Command` / `[Action][Entity]Query` — `CreateTimesheetEntryCommand`, `GetAvailabilityCalendarQuery`

Code Naming (C#):

- Classes/methods/properties: PascalCase
- Private fields: `_camelCase`
- Local variables/parameters: camelCase
- Constants: PascalCase (not ALL_CAPS)
- Interfaces: `I[Name]` — `IStaffingRecommendationService`

Code Naming (TypeScript/React):

- Components: PascalCase — `TimesheetEntryForm`, `AvailabilityCalendar`
- Component files: PascalCase `.tsx` — `TimesheetEntryForm.tsx`
- Hook files: camelCase — `useTimesheetEntries.ts`
- Utility/lib files: camelCase — `dateUtils.ts`
- Zustand stores: camelCase files — `timesheetStore.ts`; hook export `useTimesheetStore`

Structure Patterns

Backend Project Structure (Clean Architecture):

```
Ops.Domain/
  Entities/          # Pure domain models, no EF dependencies
  Events/            # Domain event definitions ([Entity][PastTenseVerb])
  Interfaces/        # Repository and service contracts
Ops.Application/
  Commands/          # [Action][Entity]Command + Handler pairs
  Queries/           # [Action][Entity]Query + Handler pairs
  Events/            # Domain event handlers
  Validators/        # FluentValidation validators per Command
Ops.Infrastructure/
  Persistence/       # EF Core DbContext, migrations, repositories
  Outbox/            # Outbox table, Hangfire worker, retry logic
  AuditLog/          # Audit log writer (INSERT-only)
```

```
    FileStorage/       # Receipt upload + malware scan pipeline
    Auth/              # JWT validation, SSO OIDC middleware
  Ops.WebApi/
    Controllers/       # Thin — delegate to MediatR, no business logic
    Middleware/        # RBAC policy, error handling, problem details
    Program.cs         # Composition root
```

Frontend Project Structure (Feature-based):

```
src/
  features/
    timesheet/       # Components, hooks, types scoped to module
    expenses/
    projects/
    staffing/
    pto/
    clients/
    reporting/
    invoices/
    admin/
  components/        # Shared/reusable components only
  stores/            # Zustand store files (one per feature domain)
  lib/
    api/             # TanStack Query hooks + API client
    utils/           # Pure utility functions
  routes/            # TanStack Router route definitions
  types/             # Shared TypeScript types and interfaces
```

Test co-location:

- Backend: `[Project].Tests/` mirror folder structure (unit) + `[Project].Integration.Tests/`
- Frontend: `*.test.ts` / `*.test.tsx` co-located alongside source file

Format Patterns

API Response Format:

- Success: direct payload — `200 OK` returns the resource directly
- Created: `201 Created` with `Location` header + created resource body
- No content: `204 No Content` for deletes
- Errors: RFC 7807 Problem Details — `application/problem+json`

```
{
  "type": "https://tools.ietf.org/html/rfc7807",
  "title": "Validation Error",
  "status": 422,
  "detail": "Hours worked must be between 0.25 and 24",
  "instance": "/api/v1/timesheet-entries"
}
```

Paginated Responses:

```
{ "items": [], "totalCount": 0, "page": 1, "pageSize": 25 }
```

Date/Time format: ISO 8601 UTC strings throughout — `"2026-03-05T14:30:00Z"`. Never Unix timestamps. Frontend displays in local time via `Intl.DateTimeFormat`.

Booleans: `true`/`false` — never `1`/`0` or `"yes"`/`"no"`

Communication Patterns

Domain Event Naming (Outbox):

- Convention: `[Entity][PastTenseVerb]` — `EmployeeAssigned`, `PtoApproved`, `SowConfirmed`, `TimesheetEntryChanged`, `InvoiceGenerated`
- Payload must always include: `EntityId`, `OccurredAt` (UTC), `TriggeredByUserId`

TanStack Query Key Convention:

```
// Pattern: ['resource', filters-object]
['timesheet-entries', { employeeId, weekStartDate }]
['availability-calendar', { month, year }]
['projects', { clientId }]
```

Zustand Store Pattern:

- One store per feature domain
- Store contains: data slice + UI state (isModalOpen, selectedId, etc.)
- No server state in Zustand — server state lives in TanStack Query only

Process Patterns

Error Handling:

- Backend: throw typed domain exceptions (`NotFoundException`, `ValidationException`, `ForbiddenException`); global middleware catches and maps to RFC 7807 responses
- Frontend: TanStack Query `onError` → push to Zustand `notificationStore`; toast displayed by global `<NotificationQueue />` component
- Route-level error boundaries via TanStack Router `errorComponent` per route

Loading States:

- Use skeleton components (not spinners) for initial page/section loads
- TanStack Query `isPending` for data-fetching states
- Local `isSubmitting` boolean in Zustand for form mutation states
- Never block the full page — load sections independently

Validation:

- Backend: FluentValidation on every Command; validation errors return `422` with RFC 7807 body listing all field errors
- Frontend: mirror validation rules client-side for immediate feedback; never rely solely on API validation for UX

Enforcement Guidelines

All agents MUST:

- Never put business logic in API controllers — controllers call `ISender` (MediatR) only
- Never access the DB directly from a controller or application handler — always through repository interfaces
- Never enforce authorization in the UI only — every protected action requires a server-side policy check
- Never write directly to audit log tables from application code — only through `IAuditLogWriter` infrastructure service
- Never mutate server state directly in a Zustand store — server state belongs in TanStack Query cache
- Always use ISO 8601 UTC for all date/time values in API contracts — always `DateTime.UtcNow`, never `DateTime.Now`

Anti-Patterns:

- ❌ Fat controllers with business logic
- ❌ `DbContext` injected directly into controllers or application handlers
- ❌ UI-only RBAC checks
- ❌ Spinner blocking entire page for data loads
- ❌ Storing API response data in Zustand alongside UI state
- ❌ `DateTime.Now` in any API or DB operation

Project Structure & Boundaries

Complete Project Directory Structure

Root:

```
MIS321-GP2/
├── client/                  # React 19 SPA (frontend)
├── server/                  # .NET 9 Clean Architecture API (backend)
├── .gitignore
└── README.md
```

Frontend (`client/`):

```
client/
├── package.json
├── vite.config.ts
```

```
    ├── tsconfig.json
    ├── tailwind.config.ts
    ├── .env
    ├── .env.example
    ├── index.html
    ├── public/
    │   └── assets/
    └── src/
        ├── main.tsx
        ├── App.tsx
        ├── routes/
        │   ├── __root.tsx                  # Root layout (nav, auth guard)
        │   ├── index.tsx                   # Dashboard
        │   ├── timesheet/
        │   │   ├── index.tsx
        │   │   └── $weekStartDate.tsx
        │   ├── expenses/
        │   │   └── index.tsx
        │   ├── projects/
        │   │   ├── index.tsx
        │   │   └── $projectId.tsx
        │   ├── staffing/
        │   │   ├── index.tsx               # Availability calendar
        │   │   └── needs.tsx               # Staffing needs board
        │   ├── pto/
        │   │   └── index.tsx
        │   ├── clients/
        │   │   ├── index.tsx
        │   │   └── $clientId.tsx
        │   ├── reporting/
        │   │   └── index.tsx
        │   ├── invoices/
        │   │   └── index.tsx
        │   └── admin/
        │       ├── users.tsx
        │       ├── migration.tsx
        │       └── settings.tsx
        ├── features/
        │   ├── timesheet/                  # FR5–FR10
        │   │   ├── TimesheetWeekView.tsx
        │   │   ├── TimesheetEntryForm.tsx
        │   │   ├── TimesheetAuditLog.tsx
        │   │   ├── useTimesheetEntries.ts
        │   │   └── types.ts
        │   ├── expenses/                   # FR11–FR14
        │   │   ├── ExpenseList.tsx
        │   │   ├── ExpenseForm.tsx
        │   │   ├── ReceiptUpload.tsx
        │   │   ├── useExpenses.ts
        │   │   └── types.ts
        │   ├── projects/                   # FR15–FR19
        │   │   ├── ProjectList.tsx
        │   │   ├── ProjectDetail.tsx
        │   │   ├── ProjectBudgetPanel.tsx
```

```
│   │   ├── TeamMemberList.tsx
│   │   ├── useProjects.ts
│   │   └── types.ts
│   ├── staffing/                   # FR20–FR26
│   │   ├── AvailabilityCalendar.tsx
│   │   ├── AvailabilityCell.tsx
│   │   ├── StaffingNeedsBoard.tsx
│   │   ├── StaffingNeedForm.tsx
│   │   ├── useAvailability.ts
│   │   ├── useStaffingNeeds.ts
│   │   └── types.ts
│   ├── ai-staffing/                # FR27–FR28
│   │   ├── CandidateRankingPanel.tsx
│   │   ├── CandidateCard.tsx
│   │   ├── useStaffingRecommendations.ts
│   │   └── types.ts
│   ├── pto/                        # FR29–FR31
│   │   ├── PtoRequestForm.tsx
│   │   ├── PtoRequestList.tsx
│   │   ├── PtoApprovalQueue.tsx
│   │   ├── usePtoRequests.ts
│   │   └── types.ts
│   ├── clients/                    # FR32–FR35
│   │   ├── ClientList.tsx
│   │   ├── ClientDetail.tsx
│   │   ├── ClientForm.tsx
│   │   ├── useClients.ts
│   │   └── types.ts
│   ├── reporting/                  # FR36–FR40
│   │   ├── PersonalReport.tsx
│   │   ├── TeamReport.tsx
│   │   ├── OrgReport.tsx
│   │   ├── ClientReport.tsx
│   │   ├── ReportFilters.tsx
│   │   ├── useReports.ts
│   │   └── types.ts
│   ├── invoices/                   # FR41–FR44
│   │   ├── InvoiceList.tsx
│   │   ├── InvoiceGenerator.tsx
│   │   ├── InvoiceDetail.tsx
│   │   ├── useInvoices.ts
│   │   └── types.ts
│   └── admin/                      # FR1–FR4, FR45–FR46
│       ├── UserManagement.tsx
│       ├── RoleAssignment.tsx
│       ├── MigrationImport.tsx
│       ├── MigrationReview.tsx
│       ├── useUsers.ts
│       └── types.ts
├── components/
│   ├── ui/                         # shadcn/ui primitives
│   ├── layout/
│   │   ├── AppShell.tsx
│   │   ├── Sidebar.tsx
```

```
    │   │   └── TopNav.tsx
    │   ├── NotificationQueue.tsx
    │   └── SkeletonLoader.tsx
    ├── stores/
    │   ├── timesheetStore.ts
    │   ├── staffingStore.ts
    │   ├── notificationStore.ts        # Event propagation alerts
    │   └── authStore.ts                # Current user + role
    ├── lib/
    │   ├── api/
    │   │   ├── client.ts               # Base client + JWT interceptor
    │   │   └── queryClient.ts          # TanStack QueryClient config
    │   └── utils/
    │       ├── dateUtils.ts
    │       ├── rbacUtils.ts
    │       └── validationUtils.ts
    └── types/
        ├── auth.ts
        └── shared.ts                   # PaginatedResponse<T>, ApiError
```

Backend (`server/`):

```
server/
├── Ops.sln
├── Ops.Domain/
│   ├── Entities/
│   │   ├── Employee.cs
│   │   ├── TimesheetEntry.cs
│   │   ├── Expense.cs
│   │   ├── Project.cs
│   │   ├── ProjectAssignment.cs
│   │   ├── StaffingNeed.cs
│   │   ├── AvailabilityStatus.cs
│   │   ├── PtoRequest.cs
│   │   ├── Client.cs
│   │   ├── Invoice.cs
│   │   └── InvoiceLineItem.cs
│   ├── Events/
│   │   ├── EmployeeAssigned.cs
│   │   ├── SowConfirmed.cs
│   │   ├── PtoApproved.cs
│   │   ├── TimesheetEntryChanged.cs
│   │   └── InvoiceGenerated.cs
│   └── Interfaces/
│       ├── Repositories/
│       └── Services/
│           ├── IStaffingRecommendationService.cs
│           ├── IAuditLogWriter.cs
│           └── IFileStorageService.cs
├── Ops.Application/
│   ├── Commands/
│   │   ├── Timesheets/                 # FR5–FR8
```

```
│   │   ├── Expenses/                    # FR11–FR13
│   │   ├── Projects/                    # FR15–FR16
│   │   ├── Staffing/                    # FR21–FR26
│   │   ├── Pto/                         # FR29–FR31
│   │   ├── Clients/                     # FR32
│   │   ├── Invoices/                    # FR41–FR42
│   │   ├── Users/                       # FR2–FR3
│   │   └── Migration/                   # FR45–FR46
│   ├── Queries/
│   │   ├── Timesheets/                  # FR9–FR10
│   │   ├── Expenses/
│   │   ├── Projects/                    # FR17–FR19
│   │   ├── Staffing/                    # FR20, FR27–FR28
│   │   ├── Pto/
│   │   ├── Clients/                     # FR33–FR35
│   │   ├── Reports/                     # FR36–FR40
│   │   └── Invoices/                    # FR43–FR44
│   ├── Events/
│   │   ├── EmployeeAssignedHandler.cs  # → sets Soft Booked (FR21)
│   │   ├── SowConfirmedHandler.cs      # → sets Fully Booked (FR22)
│   │   ├── PtoApprovedHandler.cs       # → marks PTO days + conflict detect
(FR23–FR24)
│   │   └── TimesheetEntryChangedHandler.cs
│   └── Validators/
├── Ops.Infrastructure/
│   ├── Persistence/
│   │   ├── OpsDbContext.cs
│   │   ├── Migrations/
│   │   └── Repositories/
│   ├── Outbox/
│   │   ├── OutboxMessage.cs
│   │   ├── OutboxWriter.cs
│   │   └── OutboxWorker.cs             # Hangfire background job
│   ├── AuditLog/
│   │   ├── AuditLogEntry.cs
│   │   └── AuditLogWriter.cs           # INSERT-only writer
│   ├── FileStorage/
│   │   ├── LocalFileStorageService.cs
│   │   └── MalwareScanMiddleware.cs
│   ├── Auth/
│   │   ├── SsoAuthenticationService.cs
│   │   └── JwtTokenValidator.cs
│   └── AI/
│       └── StaffingRecommendationService.cs
├── Ops.WebApi/
│   ├── Controllers/
│   │   ├── TimesheetEntriesController.cs
│   │   ├── ExpensesController.cs
│   │   ├── ProjectsController.cs
│   │   ├── StaffingController.cs
│   │   ├── PtoController.cs
│   │   ├── ClientsController.cs
│   │   ├── ReportsController.cs
│   │   ├── InvoicesController.cs
```

```
│   │   ├── UsersController.cs
│   │   └── MigrationController.cs
│   ├── Middleware/
│   │   ├── RbacPolicyMiddleware.cs
│   │   └── ProblemDetailsMiddleware.cs
│   ├── appsettings.json
│   ├── appsettings.Development.json
│   └── Program.cs
├── Ops.Tests/
└── Ops.Integration.Tests/
```

Architectural Boundaries

API Boundary:

- All client → server communication via `/api/v1/` REST endpoints
- Every request carries JWT bearer token validated by `JwtTokenValidator`
- Policy middleware enforces role capability check before handler executes
- Controllers contain zero business logic — `ISender.Send(command/query)` only

Data Access Boundary:

- `OpsDbContext` is only accessible inside `Ops.Infrastructure`
- Application layer accesses data exclusively through repository interfaces
- Audit log tables: only `AuditLogWriter` may INSERT; no other code path touches them

Event Boundary:

- State-changing commands write domain events to `OutboxMessages` in the same DB transaction
- `OutboxWorker` (Hangfire) dispatches events to Application event handlers
- Frontend notified via TanStack Query polling on affected resources — no WebSocket at MVP

File Storage Boundary:

- All file writes go through `IFileStorageService`
- Malware scan is synchronous at upload — file rejected before storage if scan fails
- Files served via `/api/v1/files/{id}` with auth check

Requirements to Structure Mapping

FR Category	Backend Location	Frontend Location
Auth & User Mgmt (FR1–4)	`Commands/Users/`, `Auth/`	`features/admin/`, `stores/authStore.ts`
Timesheet (FR5–10)	`Commands/Timesheets/`, `Queries/Timesheets/`	`features/timesheet/`
Expenses (FR11–14)	`Commands/Expenses/`, `FileStorage/`	`features/expenses/`

FR Category	Backend Location	Frontend Location
Project Mgmt (FR15–19)	`Commands/Projects/`, `Queries/Projects/`	`features/projects/`
Staffing & Availability (FR20–26)	`Commands/Staffing/`, `Events/*Handler.cs`	`features/staffing/`
AI Staffing (FR27–28)	`AI/StaffingRecommendationService.cs`	`features/ai-staffing/`
PTO (FR29–31)	`Commands/Pto/`, `Events/PtoApprovedHandler.cs`	`features/pto/`
Client Mgmt (FR32–35)	`Commands/Clients/`, `Queries/Clients/`	`features/clients/`
Reporting (FR36–40)	`Queries/Reports/`	`features/reporting/`
Invoices (FR41–44)	`Commands/Invoices/`, `Queries/Invoices/`	`features/invoices/`
Data Migration (FR45–46)	`Commands/Migration/`	`features/admin/MigrationImport.tsx`

Data Flow

```
User Action (React)
  → TanStack Query mutation / route navigation
  → API client (JWT header injected)
  → Controller (auth check → policy check)
  → MediatR ISender.Send()
  → Command/Query Handler
  → Repository (EF Core → SQL Server)
  → [If state-changing] OutboxWriter writes event in same transaction
  → Response returned to client
  → OutboxWorker picks up event → Application event handler runs
  → Downstream state updated (availability, conflict detection, etc.)
  → Frontend TanStack Query refetch on next poll / invalidation
```

Architecture Validation Results

Coherence Validation ☑

Decision Compatibility:

- React 19 + Vite + TanStack Router + TanStack Query + Zustand + TailwindCSS — fully compatible, no version conflicts; all actively maintained
- .NET 9 + EF Core + SQL Server + Hangfire + MediatR + FluentValidation + JWT — proven enterprise combination, all packages support .NET 9
- RFC 7807 Problem Details natively supported in .NET 9 via `IProblemDetailsService` — no additional library needed
- Hangfire + transactional outbox (ADR-01) is a well-established .NET pattern; Hangfire's SQL Server storage integrates cleanly with EF Core migrations
- Clean Architecture layer boundaries enforced by project reference structure — Infrastructure references Domain; Domain has no external dependencies

Pattern Consistency:

- CQRS (Commands write, Queries read) directly supports the performance split between write-heavy timesheet operations and read-heavy reporting — aligns with ≤5s report query NFR
- TanStack Query polling replaces WebSocket for event notification at MVP. The ≤5s propagation SLA applies to server-side propagation (outbox → handler → DB update); frontend polling interval set to ≤10s for availability calendar to keep UX acceptable
- INSERT-only audit log tables enforced by DB user permissions — cannot be accidentally bypassed

Structure Alignment:

- Every FR category has a named backend directory and frontend feature folder — no orphan requirements
- Outbox, AuditLog, FileStorage, and AI are isolated infrastructure concerns — swap-out friendly

Requirements Coverage Validation

Functional Requirements — 46/46 covered ☑

All 11 FR categories explicitly mapped to backend + frontend locations. Key cross-cutting FRs:

- FR4 (RBAC enforcement) → `RbacPolicyMiddleware.cs` + service-layer ownership checks
- FR8 (tamper-evident audit trail) → `AuditLogWriter.cs` + INSERT-only DB permissions
- FR24 (conflict detection) → `PtoApprovedHandler.cs` domain event handler
- FR27–28 (AI staffing + fallback) → `StaffingRecommendationService.cs` behind `IStaffingRecommendationService`

Non-Functional Requirements Coverage:

NFR	Architecture Support	Status
Page loads ≤2s	TanStack Query cache + skeleton loaders	☑
Calendar render ≤3s	IMemoryCache snapshot + paginated fetch	☑
Event propagation ≤5s	Hangfire near-realtime dispatch (typically <1s server-side)	☑

NFR	Architecture Support	Status
Report queries ≤5s (5yr data)	EF Core + SQL Server indexed queries via CQRS read path	⚠ indexes TBD in first migration
Security (TLS, RBAC, audit, scan)	Reverse proxy TLS + policy middleware + INSERT-only tables + MalwareScanMiddleware	☑
99.5% uptime	Single deployable reduces failure surface; Hangfire retry for events	☑
70–100 concurrent users	Single server well within capacity at this scale	☑
Horizontal scaling to 200+	Docker-ready; SQL Server scales independently; IMemoryCache → Redis migration path noted	☑
30-min session timeout	JWT middleware idle expiry config	☑

Gap Analysis

Critical Gaps: 0

Important Gaps (address in first implementation sprint):

1. **Reporting query indexes** — composite indexes on `TimesheetEntries (EmployeeId, Date)`, `TimesheetEntries (ClientId, Date)`, `Expenses (ProjectId, Date)` must be defined in the first EF Core migration to meet the ≤5s report query NFR at scale
2. **Frontend polling interval** — TanStack Query refetch interval for `availability-calendar` and `staffing-needs` queries must be set to ≤10s

Nice-to-Have:

- Hangfire dashboard access restricted to Admin role
- DB connection pooling documented in connection string config

Architecture Completeness Checklist

Requirements Analysis

- ☑ Project context thoroughly analyzed
- ☑ Scale and complexity assessed (Medium-High, 70–100 users)
- ☑ Technical constraints identified (JS + C#, SQL Server, SSO)
- ☑ Cross-cutting concerns mapped (events, RBAC, audit, files, AI)

Architectural Decisions

- ☑ 4 ADRs pre-committed (event propagation, RBAC, audit trail, AI layer)
- ☑ Technology stack fully specified with versions
- ☑ Integration patterns defined (outbox, REST, TanStack Query)
- ☑ Deferred decisions documented (infra/deployment)

Implementation Patterns

- ☑ Naming conventions established (DB, API, C#, TypeScript)
- ☑ Structure patterns defined (Clean Architecture + feature-based frontend)
- ☑ Communication patterns specified (event naming, query keys, Zustand)
- ☑ Process patterns documented (error handling, loading states, validation)
- ☑ Enforcement guidelines + anti-patterns listed

Project Structure

- ☑ Complete directory tree defined (frontend + backend)
- ☑ All 11 FR categories mapped to specific files/directories
- ☑ Integration points + data flow documented
- ☑ Component boundaries established

Architecture Readiness Assessment

Overall Status: READY FOR IMPLEMENTATION

Confidence Level: High — 0 critical gaps; 2 important gaps are first-sprint items, not blockers.

Key Strengths:

- Transactional outbox fully specified with a proven pattern — highest-risk item resolved
- All 46 FRs have a physical home in the directory structure — no implementation ambiguity
- Two-layer RBAC and INSERT-only audit make security violations structurally difficult
- CQRS read/write split naturally enforced by Clean Architecture layer structure

Areas for Future Enhancement:

- Phase 2: Extract AI scoring to standalone service, Redis distributed cache, QuickBooks adapter
- Phase 3: WebSocket for real-time calendar (replace polling)
- Pre-deployment: CI/CD pipeline, hosting target, structured logging destination

Implementation Handoff

First Implementation Priorities (in order):

1. Scaffold both projects — `dotnet new cleanarch --name Ops` + clone modern-react-template
2. Configure SQL Server + EF Core initial migration (all entity tables + OutboxMessages + AuditLog tables + reporting indexes)
3. SSO JWT middleware + RBAC policy definitions
4. Transactional outbox infrastructure (Hangfire worker + OutboxWriter)
5. Build domain modules against established infrastructure